

CampusAI: An AI-Enhanced Learning Management System Module for Automated Academic Request Processing Using Retrieval-Augmented Generation

Mahak Khan

*Department of Computer Sciences
Bahria University Lahore Campus
Lahore, Pakistan
03-135231-018*

Neha Asif

*Department of Computer Sciences
Bahria University Lahore Campus
Lahore, Pakistan
03-135231-034*

Dr. Junaaid Nasir

*Department of Computer Sciences
Bahria University Lahore Campus
Lahore, Pakistan
Supervisor*

Abstract—Traditional Learning Management Systems (LMS) in higher education institutions are largely optimized for course content delivery and assessment, while academic and administrative workflows such as graduation clearance, supervisor allocation, and scholarship processing continue to rely on manual paperwork and fragmented communication channels. At Bahria University, this dependency results in delayed responses, poor request traceability, and increased administrative overhead for students, faculty, and staff. This paper presents CampusAI, an AI-enhanced LMS module that introduces a unified, intelligent request-management layer on top of existing institutional systems. The core contribution of CampusAI is a custom-developed Institutional AI Agent built using a Retrieval-Augmented Generation (RAG) pipeline, trained on the Bahria University Student Handbook and institutional policy documents, to deliver accurate and context-aware guidance on academic regulations and elective course selection. The system is built on a React.js/Next.js frontend and a Node.js/Express.js backend, secured through JWT-based Role-Based Access Control (RBAC), and supported by a hybrid cloud data architecture combining cloud-hosted PostgreSQL for structured records with a vector database (Pinecone or ChromaDB) for semantic document retrieval. The module further automates the digital “No Dues” clearance workflow, provides a real-time supervisor slot management dashboard, and centralizes scholarship and leave applications. This paper describes the problem context, system architecture, methodology, feasibility analysis, and implementation roadmap for the proposed solution, and outlines the evaluation criteria that will be used to assess retrieval accuracy, workflow automation coverage, and user experience.

Index Terms—Learning management systems, retrieval-augmented generation, large language models, academic workflow automation, role-based access control, hybrid cloud architecture, vector databases

I. INTRODUCTION

Modern higher-education institutions increasingly depend on Learning Management Systems (LMS) to support teaching, content delivery, and academic record keeping. However, most commercial and institutional LMS platforms remain narrowly focused on course-centric functionality and provide little to no support for the broad range of academic and administrative requests that students submit throughout their degree programs [1]. At Bahria University, the existing LMS

effectively manages course content and grading, but processes such as course registration appeals, medical leave applications, special examination approvals, supervisor selection for Final Year Projects (FYP), scholarship applications, and graduation clearance remain dependent on paper forms, email chains, and in-person office visits.

This manual dependency introduces several systemic problems: (i) delayed turnaround times for time-sensitive requests, (ii) lost or misplaced submissions due to fragmented communication channels, (iii) lack of transparency and traceability for students awaiting a decision, and (iv) a disproportionate administrative burden placed on faculty and staff who must manually track, validate, and respond to requests. These inefficiencies are particularly acute for high-volume, multi-department workflows such as graduation clearance, which typically requires sign-off from the Finance Office, the Library, and departmental laboratories before a student can graduate.

To address these challenges, this paper proposes *CampusAI: AI-Enhanced LMS Module*, a modular and scalable enhancement to the university’s existing LMS infrastructure. Rather than replacing the LMS outright, CampusAI integrates an intelligent request-management layer that automates submission, routing, tracking, and approval workflows. A central contribution of this work is the Institutional AI Agent – a Retrieval-Augmented Generation (RAG) system trained exclusively on verified institutional documents, including the Student Handbook and academic policy manuals – which provides students and faculty with grounded, context-aware answers to policy questions without the hallucination risk associated with general-purpose language models [2].

The remainder of this paper is organized as follows. Section II reviews related work on LMS extensions and RAG-based educational assistants. Section III describes the problem domain and system scope. Section IV presents the proposed system architecture and objectives. Section V details the development methodology. Section VI discusses feasibility, resource planning, and risk analysis. Section VII presents the project timeline. Section VIII concludes the paper and outlines future work.

II. RELATED WORK

Research on LMS augmentation has generally followed two directions: workflow digitization for administrative processes, and the integration of conversational AI for student support.

A. Administrative Workflow Automation in Higher Education

Several studies have examined the digitization of academic administrative processes, noting that manual, paper-based clearance and approval systems in universities are a persistent source of inefficiency and student dissatisfaction [1]. Prior e-governance and university-management-system literature has proposed centralized dashboards and role-based portals to reduce processing time for administrative requests, but such systems typically operate independently of the institution's core LMS, resulting in duplicated logins, disconnected data, and inconsistent user experience.

B. Retrieval-Augmented Generation for Domain-Grounded Question Answering

Large Language Models (LLMs) have demonstrated strong general-purpose language understanding, but their tendency to generate plausible yet factually incorrect responses (hallucination) limits their reliability in high-stakes domains such as academic policy interpretation. Retrieval-Augmented Generation, introduced by Lewis et al. [2], addresses this limitation by conditioning generation on passages retrieved from a verified external knowledge base, combining the fluency of parametric language models with the factual grounding of non-parametric retrieval. Open-source foundation models such as the LLaMA family [3] have since made it feasible for smaller institutions and student teams to build and fine-tune domain-specific assistants without relying solely on proprietary APIs. Vector similarity search over document embeddings, supported by systems such as Pinecone and ChromaDB, provides the semantic retrieval backbone required for RAG pipelines operating over unstructured institutional text such as handbooks and policy manuals.

C. Conversational AI and Chatbots in Education

Beyond RAG-specific literature, a broader body of work has explored conversational agents as first-line support tools in educational settings, including FAQ chatbots for admissions offices and rule-based advising assistants for course registration. These systems typically rely on hand-authored intent-response pairs or shallow keyword matching, which makes them brittle when institutional policy documents are updated or when a query does not match a pre-defined intent. By contrast, a RAG-based agent re-retrieves relevant passages at query time, so updates to the underlying handbook or policy corpus are reflected in the agent's responses without retraining or manually re-authoring conversation flows, making it substantially more maintainable for a small student development team.

D. Security in Multi-Role Academic Systems

Because academic systems must serve students, faculty, and administrative staff with different levels of access, Role-Based Access Control (RBAC) combined with stateless JSON Web Token (JWT) authentication has become a standard approach for securing REST APIs in multi-tenant web applications [4]. This approach allows fine-grained permission boundaries to be enforced at the API layer while keeping the frontend and backend loosely coupled.

E. Positioning of This Work

Unlike prior university management systems that treat AI assistance and workflow automation as separate concerns, CampusAI combines a RAG-based Institutional AI Agent with a unified request-management layer covering graduation clearance, supervisor allocation, and scholarship processing within a single hybrid-cloud architecture. To the authors' knowledge, no similar module has been previously registered as a Final Year Project at Bahria University Lahore Campus combining a React.js/Next.js and Node.js stack with cloud-based PostgreSQL and a vector database for institutional policy retrieval. Table I summarizes this positioning relative to conventional LMS-only deployments and generic form-based e-governance portals.

TABLE I
FEATURE COMPARISON WITH CONVENTIONAL APPROACHES

Capability	Course-centric LMS	Generic e-form portal	CampusAI
Course delivery & grading	Yes	No	Compatible
Digital clearance workflow	No	Partial	Yes
Supervisor slot transparency	No	No	Yes
Policy-grounded AI guidance	No	No	Yes (RAG)
Role-based multi-department routing	Partial	Partial	Yes
Semantic document search	No	No	Yes

III. PROBLEM DESCRIPTION AND SCOPE

The existing LMS and administrative systems at Bahria University primarily support course delivery and academic monitoring but lack dedicated facilities for managing official academic requests. Students rely on fragmented channels – email, paper forms, and office visits – for processes such as clearance, supervision, scholarships, and appeals, resulting in delays, confusion, and lost submissions. Faculty and administrators similarly face difficulties due to manual tracking, inconsistent request formats, and limited visibility into request histories and trends.

A. Primary Scope

The primary scope of CampusAI is to provide a unified, intelligent platform for academic request management that remains compatible with integration into the university's existing LMS. Core components include:

- A custom-developed Institutional LLM Agent for policy interpretation and elective course guidance.
- A fully automated digital “No Dues” clearance workflow spanning Finance, Library, and Laboratory departments.
- A real-time supervisor slot management dashboard for transparent FYP allocation.
- A centralized portal for scholarships, leave applications, and general academic requests.

B. Optional Scope

Time-permitting enhancements include mobile-responsive design optimization, an advanced analytics dashboard with visual reporting for administrative insights, and multi-language and voice-enabled chatbot support.

C. Beneficiaries

Direct beneficiaries include students (simplified submission, real-time tracking, AI-guided policy support), faculty members (centralized dashboards, automated slot management, structured review workflows), and administrative staff (clearance trend monitoring, analytics, automated reporting). **Indirect beneficiaries** include university administration (data-driven decision-making), IT departments (modular, well-documented integration patterns), and future developers or institutions who may adapt the RAG-based LLM integration pattern for deployment elsewhere.

IV. PROPOSED SYSTEM ARCHITECTURE AND OBJECTIVES

A. System Objectives

The system is designed to achieve the following measurable objectives:

- 1) Develop a specialized Institutional LLM (via RAG over a vector database) that provides accurate responses grounded in Bahria University policies, targeting a response accuracy rate above 95% on a curated policy question set.
- 2) Automate the graduation clearance process to eliminate physical paperwork across Finance, Library, and Laboratory sign-offs.
- 3) Implement a transparent, dynamic supervisor-slot tracking system for fair FYP allocation.
- 4) Provide intelligent elective-course guidance using the custom LLM, informed by student academic history and handbook rules.
- 5) Centralize scholarship, leave, and special-exam request management within a single dashboard.

B. High-Level Architecture

CampusAI follows a layered, modular architecture with a clear separation between the presentation layer, business-logic layer, and data layer, as illustrated in Fig. 1. The frontend, built with React.js and Next.js, communicates with the Node.js/Express.js backend through secure RESTful APIs documented via OpenAPI/Swagger. The backend enforces JWT-based RBAC to distinguish between student, faculty, and administrator permissions. Structured data – user roles, clearance status, supervisor allocations, and scholarship records – is persisted in cloud-based PostgreSQL, while institutional documents are embedded and indexed in a vector database (Pinecone or ChromaDB) to support the RAG pipeline of the Institutional AI Agent. Digitized source documents are retained in cloud object storage (e.g., Amazon S3 or Azure Blob Storage), and Redis is used as an optional caching layer for session data and frequently accessed queries.

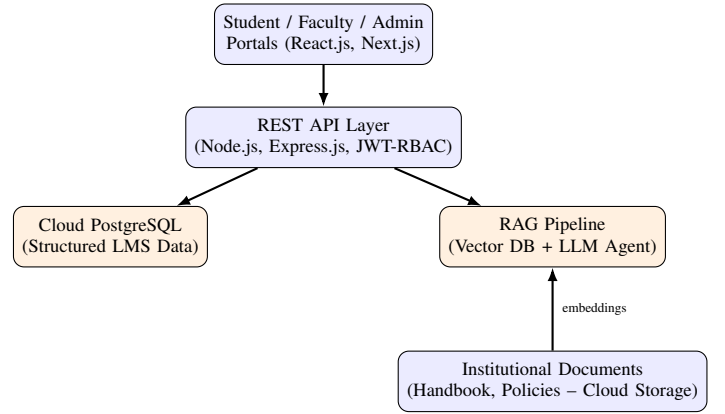


Fig. 1. High-level architecture of the CampusAI module, showing the presentation, business-logic, and data layers, including the RAG pipeline for the Institutional AI Agent.

C. Novelty

CampusAI extends beyond conventional form-filling request systems by introducing an Institutional AI Expert powered by a specialized RAG-based LLM trained on institutional documents. It is, to the authors' knowledge, the first registered FYP module at Bahria University Lahore Campus to combine a React.js/Next.js and Node.js architecture with a hybrid cloud infrastructure – cloud-based PostgreSQL for structured data and a vector database for intelligent policy retrieval.

V. METHODOLOGY

The project follows an Agile development methodology with two-week sprints, regular progress reviews, and iterative refinement based on continuous stakeholder feedback.

A. Frontend Development

The frontend is developed using React.js with the Next.js framework to produce responsive, component-based interfaces for students, faculty, and administrators. Next.js server-side

rendering improves performance and SEO, and a shared design system ensures visual and interaction consistency across portals. Client-side state is managed through the Context API or Zustand, while server-state synchronization is handled via React Query. All frontend-backend communication occurs through secure RESTful APIs.

B. Backend Development

The backend is implemented in Node.js using the Express.js framework to expose secure, scalable REST APIs. RBAC is enforced through JWT-based authentication, defining distinct permission sets for students, instructors, and administrators. API endpoints are documented using OpenAPI/Swagger specifications to support extensibility and structured integration testing.

C. Database and Cloud Architecture

CampusAI adopts a hybrid cloud-based data architecture. Cloud-based PostgreSQL serves as the primary relational store for structured LMS data, including user roles, clearance workflows, supervisor-slot management, and scholarship tracking. A vector database (Pinecone or ChromaDB) stores document embeddings for semantic retrieval, while cloud object storage secures the digitized institutional documents used to build the knowledge base. Normalization, indexing, and transactional controls ensure data integrity and performance, and Redis may be incorporated as a caching layer for session management and frequently accessed queries.

D. Knowledge Base and LLM Development

The Institutional LLM is implemented using a Retrieval-Augmented Generation approach. Official Bahria University documents – including the Student Handbook, academic policies, and procedural guidelines – are digitized and converted into embeddings stored in the vector database. Open-source base language models (e.g., the LLaMA family) are adapted so that generated responses remain strictly grounded in verified institutional knowledge, reducing the risk of hallucinated policy guidance.

E. RAG Query Processing Flow

Fig. 2 illustrates the runtime processing flow when a student or faculty member submits a policy-related query to the Institutional AI Agent. The query is first embedded and matched against the vector index of institutional documents; the top- k retrieved passages are then supplied as grounding context to the language model, which composes a response constrained to that context. This design ensures that generated answers remain traceable to a specific handbook section or policy clause, which is essential for academic accountability.

F. Integration Strategy and Development Practices

An adapter-based integration strategy demonstrates compatibility with the existing LMS while keeping the module independent of any specific LMS implementation. Version control and collaboration are managed through Git and GitHub. Comprehensive testing includes unit testing (Jest),

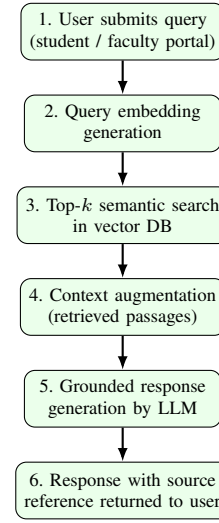


Fig. 2. Runtime query processing flow of the Institutional AI Agent's RAG pipeline.

API integration testing, and end-to-end testing (Cypress) for critical workflows. Docker containerization ensures consistent development and deployment environments across systems.

VI. FEASIBILITY ANALYSIS

A. Team Expertise

Both team members have completed coursework in Web Technologies (React.js, Node.js, Express.js, REST API development), Database Systems (cloud-based PostgreSQL), Software Engineering (design patterns, Agile methodology, system architecture), Artificial Intelligence (LLMs and RAG concepts), and Human-Computer Interaction (UI/UX and responsive design). Additional skills to be developed during the project include Next.js server-side rendering, JWT/OAuth2-based authentication, advanced React state-management patterns, PostgreSQL query optimization, and AI tooling such as LangChain for RAG pipeline orchestration.

B. Technology Stack

Table II summarizes the technology stack, all components of which are open-source, free under educational licensing, or available through low-cost cloud tiers.

C. Budget

Table III presents the estimated project budget, totaling approximately PKR 33,000, reflecting a realistic small-scale FYP budget in the Pakistani academic context.

D. Risk Analysis

Two principal risks have been identified for this project.

1) *AI Response Accuracy*: The Institutional AI Agent may occasionally provide incorrect or outdated policy information. This is mitigated by implementing RAG with source validation, restricting the knowledge base to verified university documents, scheduling regular knowledge-base updates, and

TABLE II
TECHNOLOGY STACK SUMMARY

Category	Tools / Technologies	Availability
Frontend	React.js 18+, Next.js 14, Tailwind CSS	Open source
Backend	Node.js 20+, Express.js 4, REST API, JWT	Open source
Database	Cloud-based PostgreSQL	Free (edu. tier)
AI Service	LangChain, LLaMA 3, Pinecone / ChromaDB	Custom / open source
Development	VS Code, Git, GitHub, Postman	Open source
Testing	Jest, Cypress, React Testing Library	Open source
Deployment	Vercel / Railway / Render	Free / optional
Design	Figma, Draw.io, Canva	Free / educational

TABLE III
ESTIMATED BUDGET BREAKDOWN

Item	Specification	Cost (PKR)
LLM API credits	Testing and RAG development	20,000
Cloud hosting	Frontend / backend hosting tiers	3,000
Domain name	Custom domain, annual	5,000
Miscellaneous	Fees, plugins, contingency	5,000
Total		33,000

displaying disclaimers for responses affecting critical academic decisions.

2) *LMS Integration Constraints*: Direct integration with Bahria University’s production LMS is not feasible during the development phase. This is mitigated by developing mock LMS APIs to simulate integration, maintaining clear API documentation, producing integration architecture diagrams, and designing the module to support future integration with the live LMS.

VII. PROJECT TIMELINE

The project is planned across 576 person-hours, structured into twelve milestones spanning two academic semesters (Spring and Fall 2026), as summarized in Table IV. Semester 7 covers requirement analysis, architecture and database design, and the development of the student portal, backend APIs, and supervisor/clearance modules. Semester 8 focuses on the RAG-based AI agent integration, notification and scholarship systems, comprehensive testing, deployment, user testing, and final documentation.

VIII. SECURITY, PRIVACY, AND ETHICAL CONSIDERATIONS

Because CampusAI handles sensitive academic records – including clearance status, scholarship eligibility, and supervisor

TABLE IV
KEY PROJECT MILESTONES

Milestone	Hours
Project planning & requirement analysis	22
System architecture & database design	64
LLM knowledge base & vectorization	64
Student portal development (UI & forms)	96
Backend APIs & business logic	96
AI agent integration (RAG pipeline) & core testing	70
Supervisor slot & clearance modules	56
Notification & scholarship system	40
Comprehensive testing & security hardening	32
Deployment & technical documentation	16
User testing, feedback & final refinements	16
Final documentation, presentation & report	4
Total	576

allocations – security and data-protection controls are treated as first-class design requirements rather than an afterthought. All API traffic is restricted through JWT-based RBAC so that a given role can only access endpoints and records relevant to its function; for example, a student account cannot query another student’s clearance file, and only authorized Finance, Library, or Laboratory staff accounts can approve the corresponding clearance stage. Institutional documents used to build the vector knowledge base are limited to officially published, non-personal policy material, so that the RAG pipeline does not risk surfacing individual student records in a generated response. Passwords and tokens are never stored in plaintext, and all traffic between the frontend and backend is encrypted in transit. An audit trail of clearance approvals and AI-agent responses is maintained so that decisions affecting a student’s academic standing remain traceable to a specific approver or source document, addressing the accountability concerns that motivated the disclaimer-based mitigation described in Section VI-D.

A further ethical consideration is over-reliance on AI-generated guidance for decisions that carry academic consequences. To mitigate this, the Institutional AI Agent is explicitly scoped to advisory use: responses concerning binding decisions (e.g., final clearance approval) are always confirmed by the responsible department through the workflow layer rather than being auto-approved by the language model, keeping a human decision-maker in the loop for consequential outcomes.

IX. EVALUATION PLAN

Upon completion, CampusAI will be evaluated along three dimensions. *Retrieval and answer accuracy* will be measured by curating a benchmark set of policy questions derived from the Student Handbook and comparing the Institutional AI Agent’s responses against verified ground-truth answers,

targeting the 95% accuracy objective stated in Section IV-A. *Workflow automation coverage* will be assessed by measuring the proportion of graduation-clearance, supervisor-allocation, and scholarship requests that are fully processed without manual paper-based intervention. *Usability* will be evaluated through structured user testing sessions with students and faculty, capturing task-completion time and satisfaction scores for core workflows such as request submission and clearance tracking. In addition, system-level metrics such as average end-to-end response latency of the RAG pipeline and the proportion of queries answered with a traceable source citation will be logged during the testing phase, since both factors directly affect user trust in the Institutional AI Agent’s guidance.

X. CONCLUSION AND FUTURE WORK

This paper presented CampusAI, an AI-enhanced LMS module designed to address the inefficiencies of manual academic and administrative request handling at Bahria University. By combining a RAG-based Institutional AI Agent with an automated clearance workflow, a transparent supervisor-allocation dashboard, and a centralized scholarship and leave-management portal, the proposed system aims to reduce administrative overhead, improve transparency, and accelerate request processing for students, faculty, and administrative staff. The system’s hybrid cloud architecture – combining relational storage for structured records with vector-based semantic retrieval – provides a reusable pattern for institutions seeking to integrate grounded AI assistance into existing LMS infrastructure without a full platform replacement.

Future work will focus on completing the planned implementation across the two-semester timeline, conducting the accuracy and usability evaluations described in Section VII, and exploring the optional enhancements identified in Section III-B, including mobile-responsive optimization, an administrative analytics dashboard, and multi-language or voice-enabled chatbot interaction. Beyond the scope of this project, the adapter-based integration pattern and RAG-based institutional agent design may be generalized for adoption by other higher-education institutions seeking similar administrative-automation capabilities.

REFERENCES

- [1] A. Alenezi, “Barriers to Learning Management Systems (LMS) adoption in higher education: A systematic review,” *Information Development*, 2023.
- [2] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [3] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [4] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 4, no. 3, pp. 224–274, 2001.